

Interview Prep: Backtesting Engine Project

The 60-second explanation (memorize the shape, not the words)

"I built a backtesting engine to test a momentum strategy — a moving-average crossover — on four large-cap stocks over ten years. The main thing I focused on wasn't finding a strategy that beats the market; it was building the simulation *honestly*. I specifically found and fixed a lookahead bias bug where the backtest could accidentally see a signal before it was actually available to trade on, which inflated returns from about 125% to 310% for the exact same strategy. Beyond that, I stress-tested the strategy against transaction costs and different parameter choices, built a second strategy (mean-reversion) for comparison, and added a drawdown circuit breaker. The headline result is that the strategy underperforms buy-and-hold on raw returns in this bull market, but offers meaningfully better downside protection — a real risk/return trade-off, not a failure."

The 15-second version (for when they just want a one-liner)

"A Python backtesting engine for a momentum strategy that specifically catches and quantifies lookahead bias, then stress-tests the result against transaction costs and parameter choices."

Likely questions and how to answer them

"Walk me through how your backtest actually works."

Data → signal → lagged execution → costs → equity curve → metrics vs benchmark. Emphasize the lag: "the signal computed from today's close can only be traded starting tomorrow — I enforce that with `signal.shift(1)` before anything touches returns." This is the single most important thing to be fluent in; it's your strongest, most specific technical claim.

"What is lookahead bias, and how did you find it?"

Explain the concept (simulation "seeing the future" by using data before it was actually available), then the concrete number: removing the lag inflated returns from ~125% to ~310% on the same data. If asked how you *found* it — be honest: you built it in deliberately (`.shift(1)`) based on understanding the concept first, then demonstrated its effect by running the backtest with and without it as a sanity check, not that you stumbled onto a bug by accident. (If you actually did catch a real accidental bug of this type during development — like the `strat_return` vs `transcost_return` mixup that broke your Sharpe ratio — that's an even better story. Tell that one: you noticed Sharpe was *suspiciously identical* across four different cost levels, traced it back to a variable name mixup, and fixed it. That's a real debugging story, and interviewers love those far more than "it worked first try.")

"Why moving averages? Why not something more sophisticated?"

Moving-average crossover is one of the simplest, most interpretable trend signals — the point wasn't to find a novel alpha source, it was to build a *correct, well-tested simulation framework* where the strategy itself is almost a placeholder. Simplicity was a deliberate choice so bugs in the engine wouldn't hide behind a complicated model. You then extended this by building a genuinely different strategy type (mean-reversion) to show the framework generalizes.

"Why Sharpe ratio? What does it actually measure, and what are its weaknesses?"

Sharpe = average excess return over risk-free rate, divided by volatility of returns, annualized. It measures *return per unit of risk taken* — a smoother path to the same return scores higher than a bumpy one. Weaknesses worth naming (shows depth, don't wait to be asked): it penalizes upside volatility the same as downside volatility (a strategy with big unexpected wins looks "risky" by this measure even though nobody minds that kind of risk), and it assumes returns are roughly normally distributed, which real markets often violate (fat tails, crash risk). A more complete analysis might also report Sortino ratio (downside-only volatility) or Calmar ratio (return/max drawdown).

"Why did you choose 10 bps for transaction costs? Why not 0, or 50?"

10 bps is a reasonable estimate for large-cap, liquid stocks traded at modest size, bundling commission (often ~0 today), bid-ask spread, and minor slippage. Rather than picking one number and hoping it's right, you tested sensitivity across 0/10/30/50 bps specifically *because* the "right" number is a judgment call — the interesting finding is how much the result degrades as costs rise, not defending one specific number as correct.

"Why 0% risk-free rate for Sharpe?"

Simplicity, stated as a limitation in the README rather than hidden. The real risk-free rate (historically ~2% for short-term T-bills) would modestly lower every Sharpe figure reported, but wouldn't change the *relative* ranking between strategies or cost levels, which is what the analysis actually turns on.

"Did you overfit your parameters?"

This is exactly what the parameter sensitivity test was built to answer. Sharpe stayed positive across all 6 window pairs tested (0.66–1.16), with a smooth, sensible pattern (longer windows outperforming shorter, noisier ones) rather than one lucky spike surrounded by failures — evidence against overfitting, though a genuinely rigorous test would also include true out-of-sample validation (train parameters on one period, test on a completely separate later period), which this project doesn't yet do. Say that limitation openly if asked what you'd add next.

"Why did momentum beat mean-reversion? Doesn't that mean mean-reversion is worse?"

No — it means mean-reversion was mismatched to this specific market regime.

2016–2026 was a mostly sustained bull run for these four tickers; momentum strategies are built to ride sustained trends, while mean-reversion structurally bets *against* the prevailing direction, which costs it in a trending market. Mean-reversion tends to shine in choppy, range-bound markets instead. This is a regime-dependence finding, not a strategy quality ranking — a good interview answer shows you know *when* each tool is appropriate, not just which one "won" once.

"What's the biggest limitation of this project?"

Have 2–3 ready, don't just say "none" or grasp for something trivial:

- Small, correlated universe (4 large-cap tech/finance names) — not statistically robust, and doesn't test how the strategy behaves on small caps, other sectors, or international markets
- No true out-of-sample testing — parameters were evaluated on the same data they were tested on, which risks a subtler form of overfitting than what the sensitivity grid alone catches
- No slippage/market-impact modeling beyond a flat per-trade cost, which would understate costs for larger position sizes

"How would you extend this if you had another week?"

Pull from the README's "if I had more time" section: out-of-sample testing on a different regime, a broader/less-correlated universe, short positions, volatility-scaled position sizing, a risk-parity benchmark.

"What was the hardest part to build?"

Be honest and specific — this is where a real debugging story lands well. Good options: the stateful mean-reversion "hold between entry and exit" logic (explain why a simple elementwise comparison couldn't do it — it needed to remember whether you were already holding a position from the previous day, which required a genuinely different, loop-based approach vs. the vectorized momentum signal). Or: the Sharpe ratio bug where a variable name mixup (`strat_return` vs the cost-adjusted `transcost_return`) silently broke one specific metric while everything else looked fine — and how you caught it by noticing an impossible level of precision-matching across different cost inputs.

"Is this something you'd actually trade with real money?"

No — and say so plainly, it's a sign of maturity, not weakness. This is a research/engineering exercise demonstrating rigorous backtesting practice, not a production trading system. Real deployment would need out-of-sample validation, live paper-trading, more robust risk controls, realistic slippage modeling for actual position sizes, and a much broader universe before any of these results could be trusted with capital.

Things to have ready to say out loud, cold, no notes

- Why the one-day lag exists and what breaks without it (numbers: ~125% vs ~310%)
- The Sharpe ratio formula and what it intuitively measures
- Why max drawdown matters even when total return looks good
- The momentum-vs-mean-reversion regime explanation
- At least one specific bug you personally found and fixed, with the actual cause (not just "I fixed some bugs")
- One thing you'd do differently or add next, unprompted